Code:

```
#include <stdio.h>

#include <GL/gl.h>

#include <GL/glut.h>

#include <math.h>


float X1, Y1, X2, Y2, m;

float dx, dy;


void display(void)

{

   glClear(GL_COLOR_BUFFER_BIT);

   glColor3f(1.0, 1.0, 1.0);

   glPointSize(8.0f);

   glBegin(GL_POINTS);


   dx = X2 - X1;

   dy = Y2 - Y1;


   // Handle Vertical Line (Infinite Slope)

   if (dx == 0) {

      printf("--- Vertical Line ---\n");

      int steps = abs((int)dy);

      float yStep = (Y1 < Y2) ? 1 : -1;
```

```
    float currentY = Y1;
    for(int i = 0; i <= steps; i++) {
        float rX = roundf(X1);
        float rY = roundf(currentY);
        glVertex2f(rX / 100.0, rY / 100.0);
        printf("(%.2f, %.2f) ------> Rounded -----> (%.0f, %.0f)\n", X1, currentY, rX, rY);
        currentY += yStep;
    }
}
else {
    m = dy / dx;
    printf("--- Slope (m) = %.2f ---\n", m);

    float xInc = X1, yInc = Y1;
    int steps;
    float xStep, yStep;

    // Logic for different slope conditions
    if (m < -1) {
        // Case: m < -1 (Steep Negative)
        steps = abs((int)dy);
        yStep = (Y1 < Y2) ? 1 : -1;
        xStep = dx / (float)steps;
    }
    else if (m == -1) {
```

```
    // Case: m = -1 (Perfect Negative Diagonal)
    steps = abs((int)dx);
    xStep = (X1 < X2) ? 1 : -1;
    yStep = (Y1 < Y2) ? 1 : -1;
}
else if (m > -1 && m < 0) {
    // Case: m > -1 (Shallow Negative)
    steps = abs((int)dx);
    xStep = (X1 < X2) ? 1 : -1;
    yStep = dy / (float)steps;
}
else if (m >= 0 && m <= 1) {
    // Case: Positive Shallow
    steps = abs((int)dx);
    xStep = (X1 < X2) ? 1 : -1;
    yStep = dy / (float)steps;
}
else {
    // Case: m > 1 (Positive Steep)
    steps = abs((int)dy);
    yStep = (Y1 < Y2) ? 1 : -1;
    xStep = dx / (float)steps;
}

// Execution and Printing
```

```
        for(int i = 0; i <= steps; i++) {
            float rX = roundf(xInc);
            float rY = roundf(yInc);

            // Plotting scaled values
            glVertex2f(rX / 100.0, rY / 100.0);

            // Outputting the float coordinates and their roundf result
            printf("(%.2f, %.2f) ------> Rounded -----> (%.0f, %.0f)\n", xInc, yInc, rX, rY);

            xInc += xStep;
            yInc += yStep;
        }
    }

    glEnd();
    glFlush();
}

void init(void)
{
    glClearColor(0.0, 0.0, 0.0, 1.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    // Ortho is set to 5.0 to handle coordinates up to 500 comfortably
```

```
    glOrtho(0.0, 5.0, 0.0, 5.0, -1.0, 1.0);
}

int main(int argc, char** argv)
{
    printf("Enter (X1, Y1): ");
    scanf("%f %f", &X1, &Y1);
    printf("Enter (X2, Y2): ");
    scanf("%f %f", &X2, &Y2);
    printf("\nProcessing DDA Steps:\n");
    printf("-------------------------------------------------\n");

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("DDA Line Drawing - Rounded Output");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```

Output:

```
Select "C:\Users\Computer City\Downloads\Documents\La5_task\bin\Debug\La5_task.exe"
Enter (X1, Y1): 50 20
Enter (X2, Y2): 70 -20

Processing DDA Steps:
-----------------------------------------------------
--- Slope (m) = -2.00 ---
(50.00, 20.00) ------> Rounded -----> (50, 20)
(50.50, 19.00) ------> Rounded -----> (51, 19)
(51.00, 18.00) ------> Rounded -----> (51, 18)
(51.50, 17.00) ------> Rounded -----> (52, 17)
(52.00, 16.00) ------> Rounded -----> (52, 16)
(52.50, 15.00) ------> Rounded -----> (53, 15)
(53.00, 14.00) ------> Rounded -----> (53, 14)
(53.50, 13.00) ------> Rounded -----> (54, 13)
(54.00, 12.00) ------> Rounded -----> (54, 12)
(54.50, 11.00) ------> Rounded -----> (55, 11)
(55.00, 10.00) ------> Rounded -----> (55, 10)
(55.50, 9.00) ------> Rounded -----> (56, 9)
(56.00, 8.00) ------> Rounded -----> (56, 8)
(56.50, 7.00) ------> Rounded -----> (57, 7)
(57.00, 6.00) ------> Rounded -----> (57, 6)
(57.50, 5.00) ------> Rounded -----> (58, 5)
(58.00, 4.00) ------> Rounded -----> (58, 4)
(58.50, 3.00) ------> Rounded -----> (59, 3)
(59.00, 2.00) ------> Rounded -----> (59, 2)
(59.50, 1.00) ------> Rounded -----> (60, 1)
(60.00, 0.00) ------> Rounded -----> (60, 0)
(60.50, -1.00) ------> Rounded -----> (61, -1)
(61.00, -2.00) ------> Rounded -----> (61, -2)
(61.50, -3.00) ------> Rounded -----> (62, -3)
(62.00, -4.00) ------> Rounded -----> (62, -4)
(62.50, -5.00) ------> Rounded -----> (63, -5)
(63.00, -6.00) ------> Rounded -----> (63, -6)
(63.50, -7.00) ------> Rounded -----> (64, -7)
(64.00, -8.00) ------> Rounded -----> (64, -8)
(64.50, -9.00) ------> Rounded -----> (65, -9)
(65.00, -10.00) ------> Rounded -----> (65, -10)
(65.50, -11.00) ------> Rounded -----> (66, -11)
(66.00, -12.00) ------> Rounded -----> (66, -12)
(66.50, -13.00) ------> Rounded -----> (67, -13)
(67.00, -14.00) ------> Rounded -----> (67, -14)
(67.50, -15.00) ------> Rounded -----> (68, -15)
(68.00, -16.00) ------> Rounded -----> (68, -16)
(68.50, -17.00) ------> Rounded -----> (69, -17)
(69.00, -18.00) ------> Rounded -----> (69, -18)
(69.50, -19.00) ------> Rounded -----> (70, -19)
(70.00, -20.00) ------> Rounded -----> (70, -20)
--- Slope (m) = -2.00 ---
(50.00, 20.00) ------> Rounded -----> (50, 20)
(50.50, 19.00) ------> Rounded -----> (51, 19)
```